MATOGEN INTERNSHIP

GUIDED PROJECT

FEBRUARY 2025

INTRODUCTION

Welcome to the awesome Matogen team! 😃 Thank you for deciding to do your internship with us - we are looking forward to work with you and to show you as much as we know about the IT industry so that you can be better equipped to live out your passions in life!))

Please have a look at the extract from the Matogen policy document below:

- Culture

Vision

Improving lives through software.

Mission

To make the world a better place through the software that we built.

Objectives

- To save people time and money by building awesome systems.
- To make each team member grow to become a master in his/her area of passion and expertise.
- To create sustainable happy work environments.

Core values (what we believe in)

- Quality
- Transparency
- Respect
- Honesty
- Efficiency
- Creativity

Creed (may grow later) (checklist for ourselves)

- Be happy
- Don't be greedy
- Don't be selfish
- Always put yourself in other's shoes
- Care
- Believe in yourself
- Focus more on the future than the past

Some of the things we highly value at Matogen specifically and in the IT industry in general are the skills of critical thinking, initiative and good communication. The feedback your mentor will provide you with upon the completion of this internship will be based on your performance in these areas. Your internship-resulting reference will be to a greater extent about the principle-based evaluation rather than whether or not you have completed 100% of the project tasks with the perfect coding skills (although that is also not completely disregarded).

If you struggle to understand any concepts of this project, please do not hesitate to contact your mentor as soon as you need to. Feeling inadequate to ask supposedly “dumb” questions in most cases results in going rogue on a task and spending valuable project time to search for something for hours that a senior resource could explain to you in minutes. This is bad for both our image to our clients, your image to the rest of the team and the project budget.

On the same note, since time is of the greatest value to us in the IT industry, please do not approach your mentor with generic “I have no clue” questions unless you absolutely have to. Do at least some basic research on your part - any solution or resource presented will go a long way compared to a blank page. If your mentor suggests reading a resource before giving it another go, it’s not because they are mean to you and don’t want to give you the answer - but because they want to teach you critical thinking and independence.

And finally, please be prompt and attempt to respond to all communication within one hour during working hours. Respond to all communication within 24 hours at the latest! If you are too busy to give a proper response within the first hour, please send a one-liner acknowledging the receipt of communication, with a promise to get back a little later. Good communication skills - and, ironically, not good coding skills - are what keeps customers sticking with us and happy!

BUSINESS REQUIREMENTS

A. Like any IT business, Matogen works with real customer data, which is most of the time confidential and sensitive information, protected by law. When you start with the internship with us, we will ask you to sign the Non-Disclosure Agreement, for mutual protection.
B. As mentioned earlier, time is of the highest value when working in the IT industry. Please use https://clockify.me/ to track all your hours on this project so that you can have a detailed report on how you have spent your time. Be descriptive with every item that you log against, especially when you have meetings with your mentor or other colleagues.
C. To keep communication simpler and faster, as well as all in one place, we will be using Slack. You may either use it via the web browser or via a downloadable client. Join the communication channel by following this link: https://join.slack.com/t/matogeninternship/shared_invite/zt-2jfknkayo-2HKW_jF84oFSUxn2M_snWw

TECHNICAL REQUIREMENTS

This internship project is an introductory demonstration of many real-case challenges and tasks which a software developer regularly encounters throughout their involvement with commercial projects. It involves both front-end and back-end development, user authentication, data processing and version management.

The person attempting this internship project should have the basic knowledge of the modern web application development, specifically:

- object-oriented programming, the most common data structures and JSON data format;
- SPAs (single page applications), their advantages and implementations;
- REST APIs and their request/response interactions with the front-end web pages;
- front-end and back-end project architecture - the architecture for the front-end should be for Angular and that for the back-end should be as per each intern's framework choice (although the basic flow would probably be the same throughout);
- databases and database queries (e.g. PostgreSQL, MSSQL);
- very basic token security principles (HTTPS, SSL and encryption are not part of this project)
- Basic knowledge of Git version control. (If you need more insight on how version control works, do some R&D and/or ask for help – but make sure that you have an account on https://github.com/ before you ask!!)

Note that if you do choose a fake server/API for this project, you will not learn any intricacies of server-side request management and database processing, but you then can use the extra time to go really deep into studying the front-end specs.

Internship Project:
A warehouse website with user and stock management

SUMMARY

- The website should show some basic information on its homepage so that all visitors can reference it, such as products offered, some photos, address and contact number etc.
- The website should have a function to register new users and to allow those users to log in and out. When a user registers, they need to specify whether they are an admin or a cashier.
- A user must be able to edit their name and email after logging in.
- A user must be able to add a new product by its name, type and quantity.
- A user should be able to see an additional page with a table report of all products, their categories and quantities. The page should update in real time while other users add products.
- The admin users must be able to delete any line of the tabulated products. Cashiers may not do the deleting.
- If the user is an admin, they should also see an additional page which shows a bar chart of product categories and their quantities. The page should update in real time while other users add products.
- The website should have basic security measures in place (e.g. sanitisation against XSS, prepared statements against SQL injections, authorisation of all requests) as well as well-presentable and user-readable error handling.

DETAILED DESCRIPTION OF TASKS REQUIRED

- Research front-end and back-end project architecture, give it a good thought and decide on your style *a priori*. It's ok if it is not optimal, there is no such thing as the best architecture layout and choosing the right approach for each project comes with experience))) But you should at least implement something and not just dump all your files into one heap!

- Set up the environments:
    a. For IDE: Visual Code and Visual Studio Codeis recommended, but you can use Atom or any other IDE of your choice. It will be very convenient if your IDE will have code auto-completion and intellisense, TypeScript support as well as console capabilities (so Notepad won't work unfortunately). You are welcome to ask your mentor for suggestions.
    b. For the backend: Please use C# with .NET.
    c. For the database: MySQL Server / SQLite or PostgreSQL is recommended, but you can use any database you like
    d. For the frontend: Install the latest Angular Cli via nmp in the command line (you will need NodeJs for that if you don't have it – www.nodejs.org/en). Angular has an extensive reference guide at angular.io and multiple YouTube and written tutorials, including its installation procedure.
    e. For the database, use a postgreSQL server with a pgAdmin4 GUI.
    f. For the version control: SourceTree is suggested, but you are welcome to use any repo manager of your choice (e.g. GitKraken, GithubDesktop) – or even do command line maintenance, we will not judge your genius))

1) Create a new Angular project named ‘warehouse’, implementing your choice of architecture. By default it will run on http://localhost:4200 . Launch it to confirm that your front-end is up and running.

2) Create a ‘Home’ page in Angular which will be the home page / landing page:
    a. You may use either Bootstrap or Angular Material framework for styling/UI design.
    b. The page should have three distinct elements:
        i. A horizontal navbar on top – with the webpage name on the very left and the “Register” and “Log In” buttons on the very right.
        ii. Main body where the content will be displayed
        iii. A footer with address, phone number, dummy copyright info and a current date & time clock which should update real-time (*a la* tick) in a format YYYY-mm-dd HH:mm:ss
    c. The page should be responsive – use your browser’s debugger to test the layout for tablet and mobile screens.
    d. See project requirements to decide which info should be here - you can use any fake info. The presence of this page is more important than its content so KISS))) You will have much more fun with the stuff coming up below))
    e. Implement Angular routing here - you will only be able to actually test it by step 4, but it is good if it is already set up at this stage.

3) Create a server location (something along the lines of localhost:8000), which will point to your backend API files.
    a. Implement your choice of architecture
    b. NB: Test that your server requests are correctly processed before starting the next step. You can simply use your browser or a software like Postman.
    c. For the sake of simplicity, all requests going to the server may be of the POST type.

4) Create a ‘Register’ page containing a registration form with validation (API post user):
    a. Write the user info to the database once a user is registered.
    b. Ask for name, email, password and role (dropdown).
    c. The form can be either reactive or template-driven.
    d. Enforce all fields on this form to be required with form validation.
    e. DB format: id (primary), name, password (hashed), email, role (enum: admin, cashier), token, tokenExpiry (int), lastLogin (DateTime), createdOn (DateTime), updatedOn (DateTime) .
    f. For extra points (but not mandatory) you can further increase the security of the password by salting it!
    g. Remember that each server-side request contains headers and is coming from a source too – so you need to set your server up so that it allows the origin and the headers.

5) Create a ‘Login’ page containing a login form where an existing user can log in:
    a. Edit the ‘lastLogin’, ‘token’ and ‘tokenExpiry’ database fields upon login.
    b. Test that your server requests are correctly processed before starting the next step.

c. Send the token back to the frontend and store it in the state and in the localStorage of the browser (very insecure, but request-response security is beyond the scope of this internship).
d. Now is also a good time to create a 'Log Out' button in the navbar))) It should clear all user info in the browser, erase the token and the tokenExpiry and redirect the user to the login page. And remember to hide the 'Log In' button when a user is logged in))
e. If the user is successfully logged in, make sure they are automatically redirected to their 'Profile' page (see the next step).

6) Create a 'Profile' page where a user can edit their name and email.
    a. Should be private, visible only when a user is logged in (when there is user info in the app state).
    b. Make sure that 'Login' and 'Register' pages at this stage are only visible by guests and not by those who logged in.
    c. Place the navigation for the 'Profile' page into the top navbar.
    d. Also place the name of the user next to the 'Log Out' button in the navbar so that the user can see their own name once they logged in))

7) When a user refreshes the page or closes and reopens the tab/browser, create an auto-login capability by using the token value in the browser's 'localStorage'.

8) Add the 'Add Products' page where the users can add a new product via a form.

    a. The fields to be provided are product name, product category (enum: Food, Clothes, Medicine, Household), product quantity (integers, 1-100).
    b. Place sensible validation on product quantity input.
    c. The database should get an additional table with fields of id (primary), createdOn, updatedOn and createdBy, which would be a foreign key pointing to the user record. If you implement the JSON file approach, please use a new JSON file to keep the user data and the stock data separate.

9) Add the page where the users can see all products stored so far in the database.

    a. You can use any table formatting of your choice
    b. The table headers should be dynamic, in case more columns will be added to the product database later.
    c. Each record should be deletable, but the delete functionality (e.g. button) should only be usable by the admin users.

10) Add an admin-only page to the navbar which will open a horizontal bar chart plotting product categories vs product quantities.
    a. The requirement is to place the categories on the Y-axis and the quantity on the X-axis.
    b. You may keep the generation of plotting data either on the server side or the client side.
    c. You can use any third-party plotting software of your choice (e.g. charts.js) - install it into your Angular project with npm.

CONGRATULATIONS!! YOU HAVE FINISHED THE INTERNSHIP PROJECT!
WHAT A RIDE! :)